

the following SQL query. To have the months of a year, it is necessary to summarise the table on the basis of 'year' also. The GROUP BY clause is used here to categorise all the rows/records on year values. This clause, in conjunction with the aggregation function SUM, is used to get the month-wise total freight report and return a single row for every grouped item.

```
SELECT year
, SUM(CASE WHEN month = 1 THEN freight END) Jan_frg
, SUM(CASE WHEN month = 2 THEN freight END) Feb_frg
, SUM(CASE WHEN month = 3 THEN freight END) Mar_frg
, SUM(CASE WHEN month = 4 THEN freight END) Apr_frg
, SUM(CASE WHEN month = 5 THEN freight END) May_frg
, SUM(CASE WHEN month = 6 THEN freight END) Jun_frg
, SUM(CASE WHEN month = 7 THEN freight END) Jul_frg
, SUM(CASE WHEN month = 8 THEN freight END) Aug_frg
, SUM(CASE WHEN month = 9 THEN freight END) Sep_frg
, SUM(CASE WHEN month = 10 THEN freight END) Oct_frg
, SUM(CASE WHEN month = 11 THEN freight END) Nov_frg
, SUM(CASE WHEN month = 12 THEN freight END) Dec_frg
FROM (SELECT orders.*
, EXTRACT(YEAR FROM OrderDate) year
, EXTRACT(MONTH FROM OrderDate) month
FROM Orders
) Orders
GROUP BY year
ORDER BY year
```

The pivot table output is given below:

Year	Jan_frg	Feb_frg	Mar_frg	Apr_frg	May_frg	Jun_frg	Jul_frg	Aug_frg	Sep_frg	Oct_frg	Nov_frg	Dec_frg
1996							1288.18	1397.17	1123.48	1520.6	2151.86	2798.6
1997	2239	1601.5	1888.81	2939.1	3461.4	1852.7	2458.72	3078.27	3237.05	3945.5	2008.85	3758
1998	5463.4	4272.9	5379.02	6393.6	685.08							

This query will create a report table of the monthly total freight charges of each year, sorted in the descending order of years. The above data clearly shows the pattern of monthly freight charges of each year. This report can be made more statistically viable for data analysis, if we can take the mean and variance of freight charges.

Computed pivot table: Total month-wise yearly sales

Let us consider another example of the pivot table from the same database. To generate a report of the monthly total sales for each year, we can consider the virtual table invoice view. Unlike the above example, here we need a little bit of

computation to get year-wise monthly sale values.

The structure of the invoice view is as follows:

```
SELECT Orders.ShipName,
Orders.ShipAddress,
Orders.ShipCity,
Orders.ShipRegion,
Orders.ShipPostalCode,
Orders.ShipCountry,
Orders.CustomerID,
Customers.CompanyName AS CustomerName,
Customers.Address,
Customers.City,
Customers.Region,
Customers.PostalCode,
Customers.Country,
(Employees.FirstName + ' ' + Employees.LastName) AS
Salesperson,
Orders.OrderID,
Orders.OrderDate,
Orders.RequiredDate,
Orders.ShippedDate,
Shippers.CompanyName As ShipperName,
`Order Details`.ProductID,
Products.ProductName,
`Order Details`.UnitPrice,
`Order Details`.Quantity,
`Order Details`.Discount,
(((`Order Details`.UnitPrice*Quantity*(1-Discount))/100)*100)
```

```
AS ExtendedPrice,
Orders.Freight
FROM Customers
JOIN Orders ON Customers.CustomerID = Orders.CustomerID
JOIN Employees ON Employees.EmployeeID = Orders.EmployeeID
JOIN `Order Details` ON Orders.OrderID = `Order Details`.
OrderID
JOIN Products ON Products.ProductID = `Order Details`.
ProductID
JOIN Shippers ON Shippers.ShipperID = Orders.ShipVia
```